DO NOW — RECALL

D-type flip-flops

Name: _____ Class: _____ Date: _____

1. A half adder receives A = 1 and B = 0. Calculate Sum and Carry. Explain which logic operation produces each output.

2. Apply De Morgan’s law to simplify ¬(A ∨ B).

3. Insert 40, 20, 60, 30 into an empty binary search tree. State the parent of 30 and whether 30 is its left or right child.

4. A = $0.1100_2 \times 2^3$ and B = $0.0100_2 \times 2^3$. Calculate A − B, showing the mantissa calculation and decimal result.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Sum = 1 because 1 XOR 0 = 1. Carry = 0 because 1 AND 0 = 0.

2. Model answer

¬A ∧ ¬B. Negate both inputs and change OR to AND.

3. Model answer

30 is the right child of 20: 30 is less than 40, but greater than 20.

4. Model answer

The exponents already match. $0.1100_2 - 0.0100_2 = 0.1000_2$. The result is $0.1000_2 \times 2^3 = 4_{10}$.

By the end, you can…

• state the purpose of a D-type flip-flop

• identify the D input, clock input and Q output

• trace how Q changes only when the clock triggers

Vocabulary — know these terms

Recall the term — D-type flip-flop

Recall the term — D input

Recall the term — clock

Recall the term — trigger

Recall the term — Q output

Recall the term — state

Recall the term — edge

Recall the term — register

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

If D changes but no clock trigger occurs, Q should keep its previously stored value.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

NEW LEARNING

Read, annotate and explain

Why do we need flip-flops?

Logic gates calculate; a flip-flop remembers. Registers must hold values even when other inputs change.

D is the data input. Q is the stored output. CLK determines when Q can update.

Imagine a simplified door controller. D = person detected (1) or not (0). CLK ↑ = controller samples. Q = stored sensor reading — not the motor itself.

Real door systems include safety sensors, delays and extra control logic. This is an analogy for how a flip-flop remembers a sensor value.

Connecting NOT Q back to D makes a D-type flip-flop toggle on every rising edge. Q therefore runs at half the clock frequency .

With inverted Q fed to D, Q toggles on every ↑. Fifteen divide-by-two stages can turn a 32,768 Hz watch oscillator into a 1 Hz timing signal.

↑ Rising edge: CLK goes from 0 to 1 and the flip-flop samples D. ↓ Falling edge: CLK goes from 1 to 0 and Q holds. This D flip-flop triggers on the rising edge only.

At a rising clock edge (↑) , Q copies D. Compare the old Q with D : if they differ, Q changes; if they match, Q stays the same.

No rising edge? Q keeps its old value, even if D changes or CLK falls. The arrows in the result column show the direction of Q 's change, not a different clock trigger.

Read D only when CLK has a rising edge (↑). Q is the stored output.

Between clock edges, Q holds.

Q initially stores 0. Predict Q after the first rising edge.

Read D only when CLK has a rising edge (↑). Q is the stored output.

Between clock edges, Q holds.

At ↑1, D = 1. Q becomes 1.

Read D only when CLK has a rising edge (↑). Q is the stored output.

Between clock edges, Q holds.

At ↑2, D = 0. Q becomes 0.

Read D only when CLK has a rising edge (↑). Q is the stored output.

Between clock edges, Q holds.

At ↑3, D = 1. Q becomes 1. Keep Q constant between edges.

Eight D-type flip-flops, sharing a clock, can store eight bits (one byte) simultaneously.

Why do we need flip-flops?

Logic gates calculate; a flip-flop remembers. Registers must hold values even when other inputs change.

Meet the D-type flip-flop

D is the data input. Q is the stored output. CLK determines when Q can update.

Supermarket door analogy

D can change before the sample. Q only stores the new value at CLK ↑.

Digital watch: divide by two

NOT Q fed into D causes Q to toggle at each rising edge.

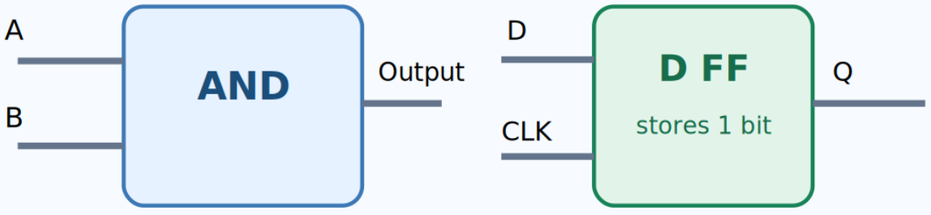
The rising edge matters

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Why do we need flip-flops?

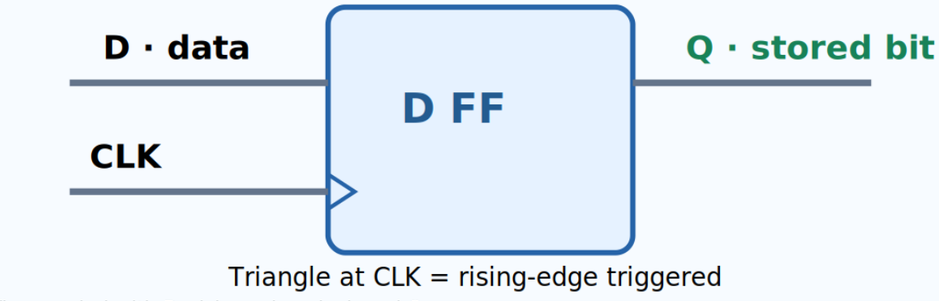
Logic gates calculate; a flip-flop remembers. Registers must hold values even when other inputs change.



Combinational AND gate versus clocked D-type flip-flop

Meet the D-type flip-flop

D is the data input. Q is the stored output. CLK determines when Q can update.



D flip-flop symbol with D, rising-edge clock and Q

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 4

ANALYSE · 4 MARKS

Q initially stores 1. D falls to 0 while CLK is LOW. The clock rises, and D returns to 1 while CLK stays HIGH. State Q after each event and explain why. [4]

Question 5

COMPARE · 3 MARKS

Explain two differences between an AND gate and a D-type flip-flop. [3]

Question 6

CHALLENGE · 4 MARKS

A 4-bit register initially stores 1010. Just before the next rising edge its D inputs are 0111. Explain the value stored before and after that edge, and what happens if the D inputs subsequently change without another rising edge. [4]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

TRACE · 4 MARKS
A rising-edge-triggered D-type flip-flop starts with Q = 0. Trace the different timing diagram and complete Q after each rising edge. [4]

Question 2

EXPLAIN · 3 MARKS
Explain the roles of the D input, clock input and Q output of a D-type flip-flop. [3]

Question 3

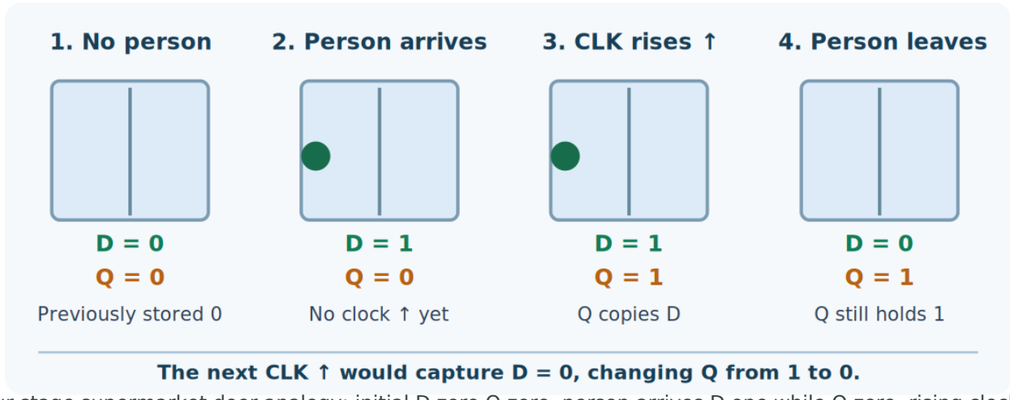
APPLICATION · 3 MARKS
A register must store an 8-bit value while combinational logic elsewhere in the CPU changes. Explain how D-type flip-flops can provide this storage. [3]

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Supermarket door: remember the sensor

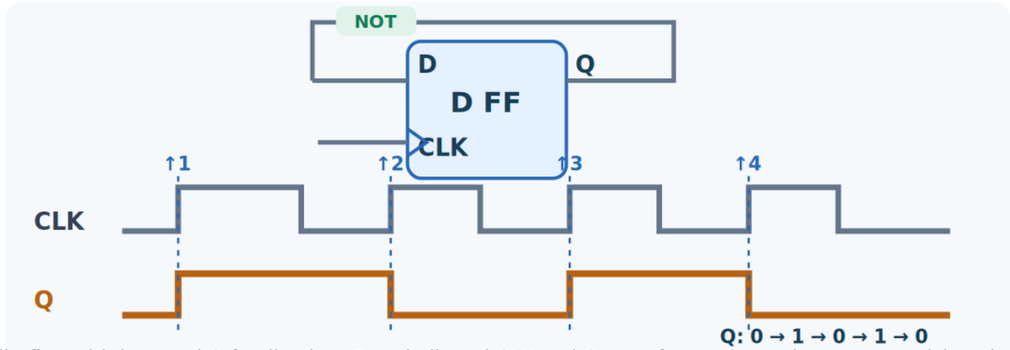
Imagine a simplified door controller. D = person detected (1) or not (0). CLK ↑ = controller samples. Q = stored sensor reading — not the motor itself.



Four stage supermarket door analogy: initial D zero Q zero, person arrives D one while Q zero, rising clock edge copies D one to Q one, sensor becomes zero but Q stays one until next rising edge.

Digital watch: halve the clock frequency

Connecting NOT Q back to D makes a D-type flip-flop toggle on every rising edge. Q therefore runs at half the clock frequency .



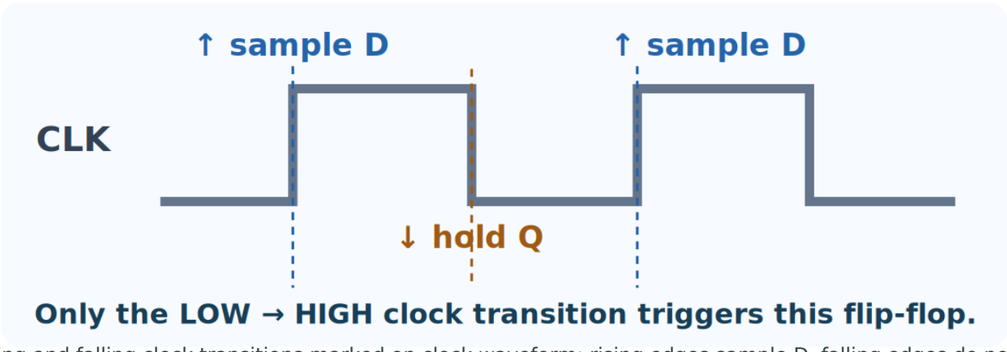
D flip-flop with inverted Q feedback to D and aligned CLK and Q waveforms. Q toggles on every rising clock edge, so two input clock cycles produce one full output cycle.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Clock edges: up or down?

↑ Rising edge: CLK goes from 0 to 1 and the flip-flop samples D. ↓ Falling edge: CLK goes from 1 to 0 and Q holds. This D flip-flop triggers on the rising edge only.



Rising and falling clock transitions marked on clock waveform; rising edges sample D, falling edges do not

When does Q actually change?

At a rising clock edge (↑) , Q copies D. Compare the old Q with D : if they differ, Q changes; if they match, Q stays the same.

Q before ↑	D at ↑	Q after ↑	Result
0	0	0	HOLDS
0	1	1	CHANGES ↑
1	0	0	CHANGES ↓
1	1	1	HOLDS

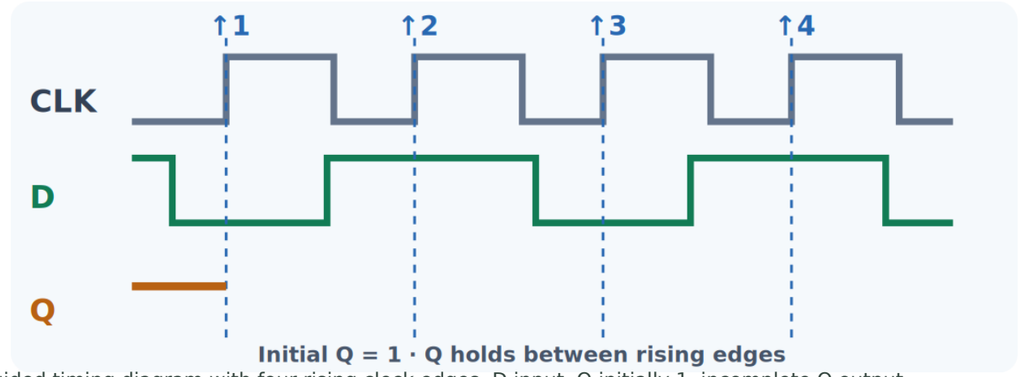
All four rising-edge cases: old Q zero with D zero stays zero; old Q zero with D one changes to one; old Q one with D zero changes to zero; old Q one with D one stays one. Without a rising edge Q holds regardless of D.

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace four rising clock edges

Q initially stores 1. Predict its value at each rising edge (↑).



Guided timing diagram with four rising clock edges, D input, Q initially 1, incomplete Q output

WE DO — GUIDED PRACTICE

Try, discuss and improve

1. Trace four rising clock edges

Q initially stores 1. Predict its value at each rising edge (↑).

Method / working:

2. Rising edge or falling edge?

Q starts at 0 . Follow the signals left to right. Which events can update Q?

Method / working:

3. Does Q follow D?

Q initially stores 1; CLK remains LOW even though D changes.

Method / working:

4. Explain one-bit storage

Use these two diagrams to explain why the output can hold a bit.

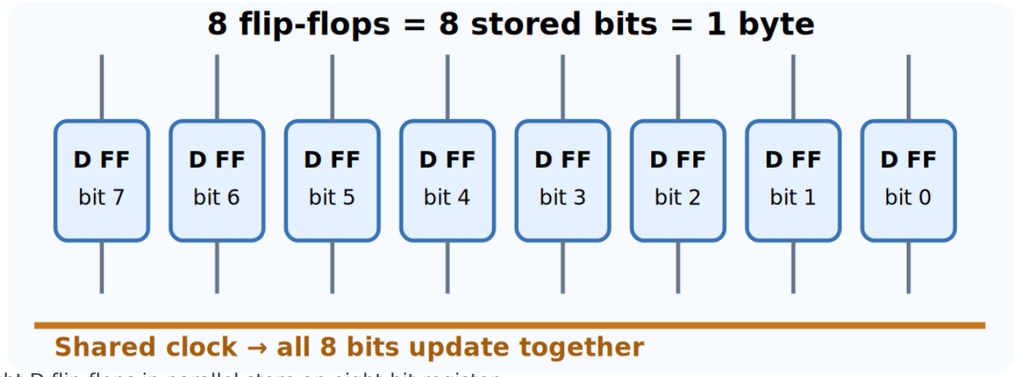
Method / working:

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Build a register

Eight D-type flip-flops, sharing a clock, can store eight bits (one byte) simultaneously.



Eight D flip-flops in parallel store an eight-bit register

NEW LEARNING

Read, annotate and explain · continued

In this lesson we use a rising-edge-triggered D flip-flop: Q copies D only when CLK changes from 0 to 1.

Between clock edges, Q holds

Changing D alone does not change Q. Without a rising edge, the flip-flop keeps its previous bit.

Locate the rising edges

Q initially stores 0. Predict Q after the first rising edge.

Trace rising edge 1

At ↑1, D = 1. Q becomes 1.

Trace rising edge 2

At ↑2, D = 0. Q becomes 0.

Trace rising edge 3

At ↑3, D = 1. Q becomes 1. Keep Q constant between edges.

Build a register

Eight D-type flip-flops, sharing a clock, can store eight bits (one byte) simultaneously.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Model when Q changes — and when it holds

Identify which edges count

Point to CLK: ↑ is LOW → HIGH and samples D. ↓ is HIGH → LOW and does not update Q. Find all three ↑ edges before reading D.

↑1: Q changes from 1 to 0

CLK rises. Read D = 0. Q was 1, so copying 0 causes a visible change. On the following falling edge Q holds 0, even though CLK moves.

D changes, but Q stays 0

Look where D rises between ↑1 and ↑2. The clock has not risen again, so Q does not follow D. The falling clock edge also leaves Q unchanged.

↑2: Q changes from 0 to 1

CLK rises again. D is now 1, and Q was 0. Sampling D makes Q change to 1. Q holds at 1 after CLK falls.

↑3: Q stays 1 — despite a rising edge

CLK rises and samples D = 1. But Q already stores 1, so the output does not visibly change. The rule is: at ↑ copy D; otherwise hold Q.